QUAKELINK

Project Development Document

Frontend | Backend | Database

Document Version	1.0.0
Project Path	/Users/bae/quakelink_app
Prepared For	QuakeLink application development and maintenance
Date	2026-07-05

เอกสารเล่มนี้สรุปภาพรวมระบบ สถาปัตยกรรม โครงสร้างไฟล์ API WebSocket และ schema ฐานข้อมูลจากโปรเจกต์ปัจจุบัน

Document Overview

เอกสารนี้จัดรูปแบบจากเอกสารพัฒนา QuakeLink ฉบับเต็ม เพื่อใช้เป็นเล่มอ้างอิงสำหรับการพัฒนา การส่งต่อโปรเจกต์ และการดูแลระบบในอนาคต

Frontend	Backend	Database
Flutter app, routing, services, realtime store, UI components	Node.js Express API, WebSocket, ThingsBoard bridge, auth, OpenAI	PostgreSQL, Prisma models, relationships, migration, data flow

สารบัญแบบย่อ

- 1. Project Overview
- 2. Tech Stack Summary
- 3. Frontend
- 4. Backend
- 5. Database
- 6. Running The Project
- 7. Verification Commands
- 8. Development Notes
- 9. Quick Reference
- 10. Document Version

เอกสารฉบับนี้สรุปการพัฒนาโปรเจกต์ QuakeLink จากโค้ดปัจจุบัน โดยแบ่งเป็น 3 ส่วนหลักคือ Frontend, Backend และ Database พร้อมโครงสร้างไฟล์, tech stack, data flow, API, วิธีรัน และข้อควรระวังในการดูแลระบบ

1. Project Overview

QuakeLink เป็นแอปติดตามและแจ้งเตือนแผ่นดินไหวแบบ real-time โดยมี Flutter เป็นฝั่งแอป และ Node.js/Express เป็น backend กลางสำหรับจัดการ authentication, WebSocket, ข้อมูลแจ้งเตือน, telemetry จาก ThingsBoard, AI chat และฐานข้อมูล PostgreSQL ผ่าน Prisma ORM

สถาปัตยกรรมโดยรวม:

```
Flutter App
  |-- REST API: auth, alerts, ThingsBoard latest, chat
  |-- WebSocket: realtime alert + sensor snapshots
  v
Node.js Express Backend
  |-- Prisma ORM
  |-- ThingsBoard API bridge
  |-- OpenAI API bridge
  |-- OAuth providers
  v
PostgreSQL Database
```

ตำแหน่งโปรเจกต์หลัก:

```
/Users/bae/quakelink_app
```

2. Tech Stack Summary

Frontend

- Framework: Flutter
- Language: Dart
- State management: Provider + ChangeNotifier
- Routing: Custom AppRouter with named routes
- HTTP client: http
- Realtime: web_socket_channel
- Maps: google_maps_flutter
- Location: geolocator, geocoding
- Storage:
- flutter_secure_storage สำหรับ access token
- shared_preferences ยังมี TokenStore อยู่ แต่ flow หลักใช้ AuthService
- Auth UI: email/password + social auth services
- Voice/AI features:
- speech_to_text
- flutter_tts
- just_audio
- Sharing/emergency actions:
- share_plus
- url_launcher

Backend

- Runtime: Node.js
- Language style: JavaScript ES Modules ("type": "module")
- Web framework: Express 5
- Realtime: ws
- ORM: Prisma 7
- Database driver/adapter: pg, @prisma/adapter-pg
- Authentication:
- JWT: jsonwebtoken
- Password hashing: bcrypt
- OAuth: Google, Facebook, GitHub
- External APIs:
- ThingsBoard REST API
- OpenAI Responses API + TTS
- Utility:
- dotenv
- axios
- xlsx สำหรับ export latency log

Database

- Database: PostgreSQL
- Schema management: Prisma
- Main models:
- User
- OAuthAccount
- Alert
- Enum:
- UserRole: PUBLIC, ADMIN

3. Frontend

3.1 Frontend Directory Structure

โครงสร้าง source สำคัญของ Flutter:

```
lib/
  main.dart
  app_router.dart

  config/
    app_config.dart
    sensor_registry.dart

  models/
    alert_event.dart
    sensor_node.dart
    sensor_snapshot.dart

  screens/
    admin_dashboard_screen.dart
    alert_details_screen.dart
    alert_screen.dart
    auth_gate.dart
    chat_ai_screen.dart
    felt_earthquake_report_screen.dart
    history_screen.dart
    home_dashboard_screen.dart
    login_screen.dart
    main_shell.dart
    map_screen.dart
    onboarding_screen.dart
    profile_edit_screen.dart
    profile_screen.dart
    register_screen.dart
    safety_guide_screen.dart
    sensor_detail_screen.dart
    settings_screen.dart
    voice_chat_screen.dart

  services/
    api_client.dart
    auth_service.dart
    chat_service.dart
    emergency_action_service.dart
    location_service.dart
    social_auth_service.dart
    token_store.dart
    voice_chat_service.dart
    ws_alert_service.dart
    ws_alert_store.dart

  widgets/
    emergency_action_grid.dart
    sensor_status_chip.dart
```

Generated/platform folders:

```
android/
ios/
```

```
web/
macos/
linux/
windows/
build/
.dart_tool/
```

หมายเหตุ:

- build/, .dart_tool/, platform generated files และ dependency cache ไม่ควรถือเป็น business source
- assets/logos/ ใช้เก็บ logo/social icon
- assets/fonts/Prompt/ ถูกอ้างใน pubspec.yaml สำหรับ font Prompt

3.2 Entry Point และ App Theme

ไฟล์หลัก:

```
lib/main.dart
```

หน้าที่:

- เริ่มแอปด้วย runApp
- ครอบแอปด้วย ChangeNotifierProvider
- สร้าง global store คือ WsAlertStore
- ตั้งค่า MaterialApp
- ใช้ AppRouter.onGenerateRoute
- route เริ่มต้นคือ /auth-gate

Theme ปัจจุบัน:

- Material 3
- colorSchemeSeed: Colors.red
- Light mode

3.3 Routing

ไฟล์:

```
lib/app_router.dart
```

Route สำคัญ:

```
/auth-gate
/login
/register
/
/onboarding
/alert
/profile
/profile-edit
/admin
/sensor_detail
/felt-report
```

แนวคิด:

- ใช้ named route แบบ centralized
- route ที่ต้องส่ง arguments เช่น:
- /alert รับ AlertScreenArgs
- /sensor_detail รับ SensorDetailArgs

3.4 Config

ไฟล์:

```
lib/config/app_config.dart
```

ใช้ --dart-define เพื่อกำหนด backend URL:

```
flutter run --dart-define=QUAKELINK_API_BASE_URL=http://127.0.0.1:8080
```

ค่า default:

```
http://127.0.0.1:8080
```

WebSocket URL ถูก derive จาก API URL:

```
http:// -> ws://
https:// -> wss://
```

ตัวอย่างเมื่อรันบนเครื่องจริง:

```
flutter run --dart-define=QUAKELINK_API_BASE_URL=http://192.168.x.x:8080
```

3.5 State Management

โปรเจกต์ใช้ provider และ ChangeNotifier

Store หลัก:

```
lib/services/ws_alert_store.dart
```

หน้าที่โดยรวม:

- เก็บรายการ alert ล่าสุด
- เก็บ sensor snapshot ล่าสุดจาก ThingsBoard
- เก็บสถานะ WebSocket
- notify UI เมื่อมีข้อมูล real-time ใหม่

Service ที่เชื่อมกับ store:

```
lib/services/ws_alert_service.dart
```

หน้าที่:

- เชื่อม WebSocket กับ backend
- รับ message type alert
- รับ message type tb_latest_all

- update WsAlertStore
- reconnect อัตโนมัติเมื่อ connection หลุด
- request sensor snapshot ครั้งแรกผ่าน WebSocket ด้วย message:

```
{"type":"get_tb_latest_all"}
```

3.6 Frontend Models

Models สำคัญ:

```
lib/models/alert_event.dart
lib/models/sensor_snapshot.dart
lib/models/sensor_node.dart
```

AlertEvent ใช้แทนข้อมูลแจ้งเตือนแผ่นดินไหว เช่น:

- alertId
- severity
- magnitude
- location
- description
- distanceKm
- lat
- lng
- nodeId
- timestamp

SensorSnapshot ใช้แทน telemetry ของ sensor เช่น:

- device name
- pga
- si
- temp
- timestamp
- location
- lat/lng
- updatedAtTs

3.7 Frontend Services

AuthService

ไฟล์:

```
lib/services/auth_service.dart
```

หน้าที่:

- register ผ่าน POST /auth/register
- login ผ่าน POST /auth/login
- save access token
- logout

- check token
- สร้าง authorization header สำหรับ API ที่ต้อง login

Storage หลัก:

```
flutter_secure_storage
```

Token key:

```
accessToken
```

ApiClient

ไฟล์:

```
lib/services/api_client.dart
```

หน้าที่:

- ดึง alerts จาก backend:

```
GET /alerts?take=...
```

- ดึง telemetry ล่าสุดจาก ThingsBoard ผ่าน backend:

```
GET /tb/latest?deviceName=...
```

/tb/latest ต้องใช้ Bearer token

WsAlertService

ไฟล์:

```
lib/services/ws_alert_service.dart
```

หน้าที่:

- เชื่อม realtime WebSocket
- parse message เฉพาะ type ที่สนใจเพื่อลดภาระเครื่อง
- reconnect แบบ progressive delay
- อัปเดต WsAlertStore

Message ที่รองรับ:

```
hello
alert
tb_latest_all
error
```

EmergencyActionService

ไฟล์:

```
lib/services/emergency_action_service.dart
```

หน้าที่:

- โทรเบอร์ฉุกเฉินผ่าน url_launcher
- share ข้อความ “ฉันปลอดภัย”

- share ตำแหน่งปัจจุบัน
- share ข้อมูล alert

ใช้ใน:

- alert screen
- alert detail
- home dashboard
- safety guide
- widget EmergencyActionGrid

ChatService และ VoiceChatService

ใช้เรียก backend สำหรับ AI chat และ voice chat:

```
POST /chat
POST /voice-chat
```

ทั้งสอง route ต้องใช้ Bearer token

3.8 Main Screens

AuthGate

หน้าตรวจ token ว่าผู้ใช้ login แล้วหรือยัง

Flow:

```
มี token -> MainShell
ไม่มี token -> LoginScreen
```

LoginScreen / RegisterScreen

หน้าลงชื่อเข้าใช้และสมัครสมาชิก

Backend routes:

```
POST /auth/login
POST /auth/register
POST /auth/google
POST /auth/facebook
POST /auth/github
```

MainShell

หน้าหลักที่รวม tab/navigation ของแอป

หน้าที่:

- เป็น shell หลัง login
- เรียกหรือจัดการหน้าหลัก เช่น Home, History, Safety, Chat, Profile

HomeDashboardScreen

หน้า dashboard หลัก:

- แสดงสถานะ monitoring
- แสดง Google Map
- แสดง sensor status
- แสดง emergency actions
- ใช้ WsAlertStore เพื่ออ่านข้อมูล sensor/alert ล่าสุด

AlertScreen

หน้าแจ้งเตือนแผ่นดินไหวแบบเต็มจอ:

- สีพื้นหลังเปลี่ยนตาม severity
- แสดง magnitude, location, distance, timestamp
- แสดงคำแนะนำ "หมอบ - กำบัง - ยึดจับ"
- มี emergency actions
- เปิด safety guide ได้

AlertDetailsScreen

หน้ารายละเอียด alert:

- แสดง severity
- แสดงข้อมูล location
- แสดง affected area map
- มีปุ่ม share alert
- มี emergency actions

SafetyGuideScreen

หน้าคู่มือความปลอดภัย:

- รองรับภาษาไทย/อังกฤษ
- ใช้งานได้แบบ offline ในแอป
- มี quick emergency actions
- มี emergency contacts

HistoryScreen

หน้าแสดงประวัติ:

- ใช้ข้อมูล alerts จาก database
- ใช้ข้อมูล sensor snapshot real-time
- เหมาะสำหรับดูเหตุการณ์ย้อนหลัง

ChatAiScreen และ VoiceChatScreen

หน้าคุยกับ AI:

- Chat text
- Voice mode
- backend จะใช้ OpenAI และ context จาก database เมื่อตรวจว่าเป็นคำถามเกี่ยวกับแผ่นดินไหว

3.9 Frontend UI Component Design

Widget สำคัญที่เพิ่มเพื่อจัด UI ให้สม่ำเสมอ:

```
lib/widgets/emergency_action_grid.dart
```

หน้าที่:

- รวมชุดปุ่มฉุกเฉินไว้ที่เดียว
- ปุ่มหลัก Call 1669 หรือ โทรฉุกเฉิน 1669 เป็น primary เต็มแถว
- ปุ่มรอง I'm safe และ Share location เป็นปุ่มคู่ขนาดเท่ากัน
- รองรับ layout narrow screen โดย stack ปุ่มแนวตั้ง
- รองรับโหมด inverted สำหรับหน้า alert ที่มีพื้นหลังสีเข้ม
- รองรับ elevated สำหรับวางเป็นการ์ดบนหน้าปกติ

หน้าที่ใช้:

```
alert_screen.dart
alert_details_screen.dart
home_dashboard_screen.dart
safety_guide_screen.dart
```

4. Backend

4.1 Backend Directory Structure

ตำแหน่ง backend:

```
server/
  .env
  .env.example
  .gitignore
  README.md
  docker-compose.yml
  package.json
  package-lock.json
  prisma.config.ts
  prismaClient.js
  server.js

  prisma/
    schema.prisma
```

สิ่งที่ไม่ควรแก้โดยตรง:

```
server/node_modules/
server/tb_latency_log.csv
server/tb_latency_log.xlsx
```

4.2 Backend Scripts

จาก server/package.json:

```
{
  "start": "node server.js",
  "dev": "nodemon server.js",
  "check": "node --check server.js",
  "prisma:generate": "prisma generate",
  "prisma:migrate": "prisma migrate dev"
}
```

คำสั่งใช้งาน:

```
cd /Users/bae/quakelink_app/server
npm install
npm run prisma:generate
npm run prisma:migrate
npm run dev
```

ตรวจ syntax:

```
npm run check
```

4.3 Environment Variables

ตัวอย่างจาก .env.example:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/quakelink
PORT=8080
JWT_SECRET=replace_with_a_long_random_secret
```

```
JWT_EXPIRES_IN=7d

OPENAI_API_KEY=

TB_BASE_URL=
TB_USERNAME=
TB_PASSWORD=
TB_DEVICE_NAME=EarthquakeSensor001
TB_DEVICE_NAMES=EarthquakeSensor001,EarthquakeSensor003,EarthquakeSensor004,EarthquakeSensor005
TB_POLL_ENABLE=false
TB_POLL_MS=3000
TB_LATESTALL_WS_ENABLE=true
TB_LATESTALL_WS_MS=5000
LATENCY_LOG_ENABLE=false

GOOGLE_CLIENT_ID_IOS=
GOOGLE_CLIENT_ID_ANDROID=
GOOGLE_CLIENT_ID_WEB=

FACEBOOK_APP_ID=
FACEBOOK_APP_SECRET=

GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
GITHUB_REDIRECT_URI=quakelink://auth/github
```

ข้อควรระวัง:

- ห้าม commit .env
- JWT_SECRET ต้องเปลี่ยนจากค่า default ก่อนใช้งานจริง
- OPENAI_API_KEY จำเป็นสำหรับ chat/voice chat
- TB_* จำเป็นสำหรับ ThingsBoard telemetry
- LATENCY_LOG_ENABLE=false เป็นค่าแนะนำสำหรับใช้งานปกติ เพื่อไม่ให้เขียนไฟล์/พิมพ์ log เกินจำเป็น

4.4 Backend Entry Point

ไฟล์:

```
server/server.js
```

หน้าที่หลัก:

- สร้าง Express app
- สร้าง HTTP server
- สร้าง WebSocket server
- เชื่อม Prisma
- เชื่อม ThingsBoard
- เชื่อม OpenAI
- จัดการ auth
- จัดการ alert persistence
- broadcast realtime data ไปยัง client

ค่า default:

```
HTTP: http://localhost:8080
WS: ws://localhost:8080
```

4.5 REST API

Health

```
GET /health
```

Response:

```
{
 "ok": true,
 "service": "quakelink-api",
 "ts": "ISO timestamp"
}
```

Local Register

```
POST /auth/register
```

Request:

```
{
 "email": "user@example.com",
 "password": "123456",
 "name": "User Name",
 "province": "Bangkok"
}
```

Validation:

- email required
- password อย่างน้อย 6 ตัวอักษร
- name required
- email ต้องไม่ซ้ำ

Response success:

```
{
 "ok": true,
 "user": {},
 "accessToken": "jwt"
}
```

Local Login

```
POST /auth/login
```

Request:

```
{
 "email": "user@example.com",
 "password": "123456"
}
```

Response success:

```
{
 "ok": true,
 "user": {},
```

```
  "accessToken": "jwt"
}
```

OAuth Login

```
POST /auth/google
POST /auth/facebook
POST /auth/github
```

Google:

- Flutter ส่ง idToken
- backend verify กับ Google audience
- upsert user และ OAuth account

Facebook:

- Flutter ส่ง accessToken
- backend ตรวจ token กับ Facebook Graph API
- upsert user และ OAuth account

GitHub:

- Flutter ส่ง code
- backend แลก access token กับ GitHub
- backend ดึง user profile/email
- upsert user และ OAuth account

Current User

```
GET /me
Authorization: Bearer <token>
```

ใช้ requireAuth middleware เพื่อตรวจ JWT

Alerts List

```
GET /alerts?take=50&skip=0
```

Response:

```
{
  "ok": true,
  "data": []
}
```

Behavior:

- เรียงจาก timestamp ล่าสุดไปเก่าสุด
- take ถูก clamp ระหว่าง 1 ถึง 200
- ไม่บังคับ auth ในโค้ดปัจจุบัน

Emit Alert

```
POST /emit
```

ใช้สำหรับส่ง alert เข้า backend, บันทึก database และ broadcast WebSocket

Required:

- magnitude
- lat
- lng
- location

Optional:

- alertId
- severity
- description
- distanceKm
- nodeId

Emit Mock Alert

```
POST /emit/mock
```

ใช้สร้าง alert ตัวอย่างสำหรับ demo/test

Mock ThingsBoard Snapshot

```
POST /mock/tb
```

ใช้สำหรับทดสอบ realtime sensor/history โดยไม่ต้องรอ ThingsBoard จริง

ThingsBoard Latest

```
GET /tb/latest?deviceName=EarthquakeSensor001
Authorization: Bearer <token>
```

Response:

```
{
  "ok": true,
  "deviceName": "EarthquakeSensor001",
  "deviceId": "...",
  "updatedAtTs": 0,
  "serverReceiveTs": 0,
  "tbApiLatencyMs": 0,
  "telemetry": {}
}
```

ThingsBoard Latest All

```
GET /tb/latest-all
Authorization: Bearer <token>
```

Response:

```
{
  "ok": true,
  "count": 4,
  "deviceNames": [],
  "latestDeviceName": "EarthquakeSensor001",
  "latestUpdatedAtTs": 0,
```

```
 "data": []
}
```

AI Chat

```
POST /chat
Authorization: Bearer <token>
```

Request:

```
{
 "message": "มีแผ่นดินไหวล่าสุดไหม"
}
```

Backend behavior:

- detect intent
- ถ้าเป็นเรื่องแผ่นดินไหว จะดึง alert ล่าสุดจาก database
- ส่ง prompt ไป OpenAI
- ส่งคำตอบกลับ frontend

Voice Chat

```
POST /voice-chat
Authorization: Bearer <token>
```

Response มี:

- replyText
- audioBase64
- mime
- intent
- sources

4.6 WebSocket

Endpoint:

```
ws://host:8080
```

เมื่อ client connect:

```
{
 "type": "hello",
 "timestamp": "ISO timestamp"
}
```

Client สามารถขอ sensor snapshot:

```
{
 "type": "get_tb_latest_all"
}
```

หรือ:

```
{
 "type": "subscribe_sensors"
}
```

Server broadcast message สำคัญ:

Alert

```
{
 "type": "alert",
 "alertId": "AL-...",
 "severity": "caution",
 "magnitude": 4.2,
 "location": "Chiang Mai",
 "description": "...",
 "distanceKm": 20.5,
 "lat": 18.7883,
 "lng": 98.9853,
 "nodeId": "node-001",
 "timestamp": "ISO timestamp"
}
```

ThingsBoard Latest All

```
{
 "type": "tb_latest_all",
 "timestamp": "ISO timestamp",
 "data": []
}
```

4.7 ThingsBoard Integration

Environment variables:

```
TB_BASE_URL
TB_USERNAME
TB_PASSWORD
TB_DEVICE_NAME
TB_DEVICE_NAMES
TB_POLL_ENABLE
TB_POLL_MS
TB_LATESTALL_WS_ENABLE
TB_LATESTALL_WS_MS
```

Telemetry keys ที่ backend อ่าน:

```
events
pga
si
temp
timestamp
id
lat
lng
location
distanceKm
```

Backend helper สำคัญ:

- tbLogin
- tbGetDeviceIdByName
- tbGetLatestTelemetry
- parseTbTelemetry

- fetchTbLatestAll
- pollTbLatestAllAndBroadcast
- pollThingsboardAndEmit

Severity derive จาก PGA/SI:

```
pga >= 1.5 หรือ si >= 80 -> severe
pga >= 0.8 หรือ si >= 40 -> moderate
อื่น ๆ -> caution
```

4.8 Latency Logging

ระบบมี latency log สำหรับทดสอบ ThingsBoard:

```
tb_latency_log.csv
tb_latency_log.xlsx
```

ปัจจุบันควรปิดไว้:

```
LATENCY_LOG_ENABLE=false
```

ถ้าต้องการ export:

```
GET /export-latency
```

ข้อควรระวัง:

- ไม่ควรเปิดตลอดเวลาระหว่างใช้งานจริง เพราะมีการเขียนไฟล์และเพิ่มภาระเครื่อง
- console log latency ถูก comment ไว้แล้ว เพื่อไม่ให้ terminal พิมพ์ตลอดเวลา

5. Database

5.1 Database Stack

- PostgreSQL
- Prisma schema:

```
server/prisma/schema.prisma
```

- Prisma client:

```
server/prismaClient.js
```

- Connection string:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/quakelink
```

5.2 Docker Database

ไฟล์:

```
server/docker-compose.yml
```

คำสั่งทั่วไป:

```
cd /Users/bae/quakelink_app/server
docker compose up -d
```

ถ้ามี error container name conflict เช่น:

```
The container name "/quakelink_db" is already in use
```

สาเหตุคือมี container เก่าชื่อเดียวกันอยู่แล้ว ต้องหยุด/ลบ/rename container เดิมก่อน หรือใช้ compose project name อื่น

5.3 Prisma Commands

Generate client:

```
cd /Users/bae/quakelink_app/server
npm run prisma:generate
```

Run migration:

```
npm run prisma:migrate
```

ถ้าขึ้น:

```
Already in sync, no schema change or pending migration was found.
```

หมายความว่า schema ใน Prisma ตรงกับ database แล้ว

5.4 Prisma Schema

Enum: UserRole

```
enum UserRole {
  PUBLIC
```

```
 ADMIN
}
```

ใช้กำหนดสิทธิ์ผู้ใช้

Model: User

```
model User {
 id           String   @id @default(cuid())
 email        String?  @unique
 passwordHash  String?
 name         String?
 province     String?
 role         UserRole @default(PUBLIC)
 lastLoginAt   DateTime? @db.Timestamptz(6)
 createdAt     DateTime @default(now()) @db.Timestamptz(6)
 updatedAt     DateTime @updatedAt @db.Timestamptz(6)

 oauthAccounts OAuthAccount[]
}
```

ใช้เก็บผู้ใช้ทั้ง local account และ social account

หมายเหตุ:

- local login จะมี passwordHash
- social login อาจไม่มี passwordHash
- email เป็น unique แต่ nullable
- role default เป็น PUBLIC

Model: OAuthAccount

```
model OAuthAccount {
 id        String   @id @default(cuid())
 provider   String
 providerId String
 email      String?
 name       String?
 avatarUrl  String?
 userId     String
 user       User     @relation(fields: [userId], references: [id], onDelete: Cascade)
 createdAt  DateTime @default(now()) @db.Timestamptz(6)

 @@unique([provider, providerId])
 @@index([userId])
}
```

ใช้เก็บ mapping ระหว่าง user ในระบบกับ provider ภายนอก เช่น:

```
google
facebook
github
```

Model: Alert

```
model Alert {
 id        String   @id @default(cuid())
 alertId    String   @unique
 severity    String
```

```
  magnitude   Float
  location    String
  description String
  distanceKm  Float?
  lat         Float
  lng         Float
  nodeId      String?

  timestamp   DateTime @default(now()) @db.Timestamptz(6)
  createdAt   DateTime @default(now()) @db.Timestamptz(6)
}
```

ใช้เก็บเหตุการณ์แจ้งเตือนแผ่นดินไหว

Behavior ฝั่ง backend:

- alertId เป็น unique
- ถ้าส่ง alertId เดิม จะ update row เดิมผ่าน upsert
- timestamp ถูกตั้งเป็นเวลาที่ server รับ alert เพื่อให้ latest เรียงถูกต้อง

5.5 Database Relationships

```
User 1 --- many OAuthAccount
Alert เป็น table อิสระ
```

OAuth relation:

- ลบ User แล้ว OAuthAccount จะถูกลบตาม เพราะ onDelete: Cascade

5.6 Data Flow: Register/Login

Register:

```
Flutter RegisterScreen
  -> AuthService.register
  -> POST /auth/register
  -> bcrypt.hash(password)
  -> prisma.user.create
  -> JWT sign
  -> Flutter save token
```

Login:

```
Flutter LoginScreen
  -> AuthService.login
  -> POST /auth/login
  -> prisma.user.findUnique(email)
  -> bcrypt.compare
  -> update lastLoginAt
  -> JWT sign
  -> Flutter save token
```

OAuth:

```
Flutter social provider
  -> backend /auth/provider
  -> verify provider token/code
  -> upsertOAuthLogin
  -> create/find User
```

```
 -> create/update OAuthAccount
 -> JWT sign
 -> Flutter save token
```

5.7 Data Flow: Realtime Sensor

```
ThingsBoard
 -> Backend pollTbLatestAllAndBroadcast
 -> WebSocket broadcast tb_latest_all
 -> Flutter WsAlertService
 -> WsAlertStore.setTbLatestAll
 -> UI rebuild
```

ค่า polling:

```
TB_LATESTALL_WS_ENABLE=true
TB_LATESTALL_WS_MS=5000
```

5.8 Data Flow: Alert

```
External source / mock / ThingsBoard poll
 -> Backend normalizeAlert
 -> saveAlertToDb
 -> Prisma upsert Alert
 -> WebSocket broadcast alert
 -> Flutter WsAlertService
 -> WsAlertStore.addAlert
 -> Alert UI / History UI update
```

5.9 Data Flow: AI Chat

```
Flutter ChatAiScreen / VoiceChatScreen
 -> ChatService / VoiceChatService
 -> POST /chat or /voice-chat
 -> backend detectIntent
 -> if quake intent: prisma.alert.findFirst latest
 -> OpenAI Responses API
 -> response to Flutter
```

Voice chat เพิ่ม:

```
OpenAI TTS
 -> mp3 buffer
 -> base64 response
 -> Flutter playback
```

6. Running The Project

6.1 Start Database

```
cd /Users/bae/quakelink_app/server
docker compose up -d
```

6.2 Setup Backend

```
cd /Users/bae/quakelink_app/server
cp .env.example .env
npm install
npm run prisma:generate
npm run prisma:migrate
npm run dev
```

Backend URL:

```
http://localhost:8080
```

WebSocket URL:

```
ws://localhost:8080
```

6.3 Run Flutter App

iOS simulator / desktop:

```
cd /Users/bae/quakelink_app
flutter pub get
flutter run --dart-define=QUAKELINK_API_BASE_URL=http://127.0.0.1:8080
```

Physical device:

```
flutter run --dart-define=QUAKELINK_API_BASE_URL=http://<LAN-IP>:8080
```

Android emulator:

```
flutter run --dart-define=QUAKELINK_API_BASE_URL=http://10.0.2.2:8080
```

7. Verification Commands

Frontend:

```
cd /Users/bae/quakelink_app
flutter analyze --no-fatal-infos --no-fatal-warnings
flutter test
```

Backend:

```
cd /Users/bae/quakelink_app/server
npm run check
```

Database:

```
cd /Users/bae/quakelink_app/server
npm run prisma:migrate
```

8. Development Notes

8.1 Files That Should Not Be Committed

```
.env
node_modules/
build/
.dart_tool/
tb_latency_log.csv
tb_latency_log.xlsx
```

8.2 Current Known Maintenance Notes

- Flutter analyzer ยังมี info/warning เดิมหลายจุด เช่น withOpacity deprecated และ unused fields บางส่วน
- Backend หลักอยู่ใน server.js ไฟล์เดียว ซึ่งเริ่มใหญ่แล้ว ในอนาคตควรแยกเป็น modules เช่น:
- routes/auth.js
- routes/alerts.js
- routes/thingsboard.js
- routes/ai.js
- ws/index.js
- services/thingsboardService.js
- services/authService.js
- TokenStore ใช้ shared_preferences แต่ auth flow หลักใช้ AuthService + flutter_secure_storage; ควรตรวจว่ามีจุดไหนยังใช้ TokenStore อยู่หรือไม่
- ควรเพิ่ม migration folder ให้ชัดเจนหากจะ deploy จริง
- ควรเพิ่ม API tests/backend tests สำหรับ auth, alert, ThingsBoard mock และ chat error path

8.3 Security Notes

- เปลี่ยน JWT_SECRET เป็น random string ที่ยาวและปลอดภัยก่อนใช้งานจริง
- ไม่เก็บ secret ใน repository
- Production ควรใช้ HTTPS/WSS
- จำกัด CORS origin ใน production
- เพิ่ม rate limit ให้ auth และ AI endpoints
- ไม่ควรเปิด /emit, /emit/mock, /mock/tb สู่ public โดยไม่มี auth หรือ admin guard
- AI endpoints มีค่าใช้จ่าย ควรเพิ่ม quota/rate limit

8.4 Production Readiness Checklist

- ตั้งค่า .env production
- ใช้ PostgreSQL production
- run migration
- ตั้ง HTTPS reverse proxy
- ตั้ง WSS
- จำกัด CORS

- เพิ่ม auth guard ให้ route ทดสอบ
- ปิด latency logging
- ตั้ง monitoring/log rotation
- ตั้ง backup database
- ตั้ง OpenAI usage limit
- ตรวจ Google/Facebook/GitHub OAuth callback ให้ตรง environment จริง
- ตรวจ mobile deep link สำหรับ GitHub OAuth

9. Quick Reference

Main Frontend Files

```
lib/main.dart
lib/app_router.dart
lib/config/app_config.dart
lib/services/auth_service.dart
lib/services/api_client.dart
lib/services/ws_alert_service.dart
lib/services/ws_alert_store.dart
lib/widgets/emergency_action_grid.dart
```

Main Backend Files

```
server/server.js
server/prismaClient.js
server/prisma/schema.prisma
server/.env.example
server/docker-compose.yml
server/package.json
```

Main API Endpoints

```
GET  /health
POST /auth/register
POST /auth/login
POST /auth/google
POST /auth/facebook
POST /auth/github
GET  /me
GET  /alerts
POST /emit
POST /emit/mock
POST /mock/tb
GET  /tb/latest
GET  /tb/latest-all
POST /chat
POST /voice-chat
GET  /export-latency
```

Main WebSocket Messages

```
hello
alert
tb_latest_all
error
```

10. Document Version

Document: QuakeLink Development Documentation
Version: 1.0.0
Generated from current local project structure
Date: 2026-07-05